

Infrastructure Seedwork 復習ガイド

「Infrastructureって何をする場所？」を、できるだけ噛み砕いて復習するためのガイドです。

1. まずInfrastructureを一言でいうと

Infrastructure = 「実際の技術につなぐ場所」です。

Domainが「業務上どうあるべきか」を決め、Applicationが「どういう順番で処理するか」を決めるのに対して、Infrastructureは「それをDBや外部APIなどを使って、どうやって実現するか」を担当します。

たとえば、Domainが「会員を保存したい」と考えていても、Domain自身は「SQLAlchemyを使おう」とは考えません。

その“保存の具体的方法”をInfrastructureが担当します。

2. 3層をざっくり覚える

層	考えること	代表例
Domain	何が正しい？	Entity / Value Object / Repository抽象 / Domain Event
Application	どう動かす？	Use Case / Command / Query / Unit of Work
Infrastructure	どうやって接続する？	SQLAlchemy / DB / Repository実装 / 外部API

覚え方：Domain = ルール、Application = 手順、Infrastructure = 技術

3. Infrastructureの仕事

- ・DBへ接続する
- ・Domainで定義されたRepositoryを、SQLAlchemyなどで実装する
- ・トランザクションを管理する
- ・認証情報などの実行環境をApplicationへ渡す
- ・外部API・メール・ストレージ・メッセージングなどへ接続する
- ・DBやORMなど、技術固有の例外を扱う

逆に、「会員登録できる条件」などの業務ルールはInfrastructureの仕事ではありません。

4. Repositoryは「何をするか」と「どう保存するか」を分ける

ここがInfrastructureを理解する重要ポイントです。

Domainでは「会員を保存できる場所がほしい」とRepositoryの抽象を定義します。

Domain側：

「Memberを保存してください」

Infrastructure側：

「ではSQLAlchemy Sessionを使ってDBへ保存します」

DomainはSQLAlchemyを知らない。

InfrastructureだけがSQLAlchemyを知っている。

これによって、DB技術を変更してもDomainの業務ルールを巻き込まずに済みます。

5. EntityとDB Modelは別物

Infrastructureでは、Domain EntityをそのままDBの都合に合わせるのではなく、Domain Entity ⇔ DB Modelの変換をInfrastructure内に閉じ込めます。

Domain Entity
↓ `_to_model()`
DB Model
↓ DBへ保存
DB

DB
↓ 読み出し
DB Model
↓ `_to_domain()`
Domain Entity

つまり、DBの形と業務の形を混ぜないことがポイントです。

6. Unit of Workって何？

Unit of Workは、簡単にいうと「一連のDB操作をひとまとまりとして扱う仕組み」です。

例えば注文確定で、注文更新・在庫更新・履歴保存を行うとします。

全部成功 → `commit`
途中で失敗 → `rollback`
最後に Session を `close`

Applicationは「トランザクションを使いたい」と考え、Infrastructureは「SQLAlchemyで実際にどう`commit/rollback`するか」を担当します。

7. Database Setup

`database\_setup.py` のようなInfrastructureの部品は、DB EngineやSession生成を担当します。

- `create_engine()`
- `sessionmaker()`
- DB接続設定

これらをDomainやApplicationから直接呼ばないことが大切です。

8. Identity Context

Applicationが「今ログインしている人は誰？」を知りたい場合、ApplicationがHTTP Requestなどを直接読むのではなく、`IdentityContext` のような抽象を使います。

HTTP Request / Test / Batch



具体的なIdentityContext



IIdentityContext



Application

これにより、Webから実行するときも、テストするときも、Applicationのコードを大きく変えずに済みます。

9. Infrastructure Exception

技術的な失敗は、Infrastructure側の例外として整理します。

例外	ざっくり意味
InfrastructureException	Infrastructure共通の例外
DatabaseConnectionError	DB接続に失敗した
MappingError	Domain ModelとDB Modelの変換に失敗した

覚え方：

- ・ 業務ルール違反 → DomainException
- ・ 入力・権限・ユースケース上の問題 → AppException
- ・ DB・ORM・外部サービスなど技術上の問題 → InfrastructureException

10. Infrastructureに業務ロジックを書かない

例えば「金額がマイナスになってはいけない」は業務ルールです。これはRepositoryやDB Adapterに書くのではなく、Domainで保証します。

Domain
Money / Order
↓
業務ルールを保証

Infrastructure
Repository
↓
保存・取得を担当

Infrastructureが判断を増やすほど、業務ルールがあちこちに散らばってしまいます。

11. 外部APIも同じ考え方

外部API、メール、S3などのストレージ、メッセージングなども基本は同じです。

Application / Domain
↓
抽象
↑
Infrastructure Adapter
↓
外部API / SDK / DB

外部SDKの型をDomainへそのまま持ち込まず、必要ならInfrastructureのAdapter内で変換します。

12. Dependency Injection (DI)

Infrastructureの具体実装をApplicationへ直接埋め込むのではなく、外から渡せる形にします。

例：

```
SqlAlchemyMemberRepository(session)
```

↓

```
UseCase(repository=repository, unit_of_work=unit_of_work)
```

こうすると、テストでは本物のDBではなくFake Repositoryなどへ差し替えやすくなります。

13. SeedworkとFeature Infrastructureの違い

`src/seedwork/infrastructure/` は、どのFeatureでも再利用できる基盤を置く場所です。

場所	置くもの
seedwork/infrastructure/	SQLAlchemy Repository基盤、Unit of Work基盤、DB設定など
features/xxx/infrastructure/	xxx機能専用のRepository、DB Model、外部API Adapterなど

Feature固有の実装をseedworkへ入れないことが重要です。

14. このプロジェクトのInfrastructure Seedwork

部品	役割	ファイル
Database	DB Engine / Session管理	database_setup.py
Repository実装	Repositoryの具体実装	sqlalchemy_repository.py
Unit of Work実装	DB Transaction	sqlalchemy_unit_of_work.py
Identity Context実装	Identity抽象を実環境へ接続	identity_context_implementation.py
Infrastructure Exception	技術固有エラー	infrastructure_exceptions.py

15. 実際の処理を追ってみる

例：「会員を登録する」

- ① Applicationが「会員登録」というUse Caseを開始
- ② Domain Entityを生成し、業務ルールをチェック
- ③ ApplicationがRepositoryの抽象を呼ぶ
- ④ InfrastructureのSQLAlchemy Repositoryが受け取る
- ⑤ Domain EntityをDB Modelへ変換
- ⑥ SQLAlchemy SessionでDBへ保存
- ⑦ Unit of Workがcommit
- ⑧ 処理完了

この流れで、DomainはDBのことを知りません。ApplicationもSQLAlchemyの細部を知りません。

16. 「これはInfrastructure？」を判断する

やりたいこと	主な担当
会員名は空文字禁止	Domain
会員登録の手順を実行	Application
MemberをDBへ保存	Infrastructure
SQLAlchemy Sessionを作る	Infrastructure
DB transactionをcommit/rollback	Infrastructure
ログインユーザー情報をHTTPから取得	外側 / Infrastructure側
「管理者だけ削除可能」というルール	Domain / Applicationの責務を整理
外部APIを呼び出す	Infrastructure Adapter

17. やってはいけない例

- Domain EntityにSQLAlchemyのSessionを持たせる
- Applicationからcreate_engine()を直接呼ぶ
- DomainでDB Modelを直接操作する
- Repositoryの中に業務ルールを書く
- 外部SDKの型をDomain Entityの属性として直接使う
- Feature専用Repositoryをseedworkへ置く

迷ったら「これは技術の話か？業務の話か？」と考えると整理しやすくなります。

18. AIにInfrastructureを作ってもらったとき

ステップ1：まず設計を相談する

- 対応するDomain / Applicationの抽象は何か？
- Repository？Unit of Work？Identity Context？外部API Adapter？
- どの技術を使うか？
- Entity ⇄ DB ModelのMappingをどうするか？
- Transactionをどう管理するか？
- どこへ配置するか？

ステップ2：合意した内容を実装する

- 既存の抽象を確認する
- Infrastructureに具体実装を追加する
- Domain/Applicationへ技術依存を漏らさない
- Mapping・Transaction・例外処理をInfrastructure内に閉じ込める
- DIでテスト時に差し替えられるようにする

19. 最後に覚える3つ

① Domain = 何が正しい？

業務ルールを守る。

② Application = どう動かす？

Use Caseの手順を組み立てる。

③ Infrastructure = どうやって実現する？

DB・ORM・外部APIなどの具体技術につなぐ。

Infrastructureは「外の世界との接続担当」。

技術をInfrastructureに閉じ込めることで、DomainとApplicationを技術から守ります。

参考：プロジェクトの INFRASTRUCTURE_RULE.md に定義された責務・配置・例外・DI・AI対応手順をもとに構成しています。